THAMES 5.1 User Guide

Jeffrey W. Bullard

2025-06-16 13:03:01-05:00

Contents

1 Software Requirements **1**
- 1.1 Putting THAMES in your path . . . 2

2 THAMES command line options **3**
- 2.1 Running THAMES non-interactively . . . 4
  - 2.1.1 Redirecting input . . . 4
  - 2.1.2 Redirecting output . . . 5

3 THAMES Input Files **5**
- 3.1 Categories of THAMES Input . . . 5
- 3.2 Microstructure initial 3D arrangement . . . 6
- 3.3 Thermodynamic data input . . . 6
  - 3.3.1 Meaning of GEMS3K the File Contents . . . 7
- 3.4 Simulation Parameters . . . 8
  - 3.4.1 Microstructure phase definition section . . . 8
  - 3.4.2 Environment section. . . . 8
  - 3.4.3 Microstructure phase definitions . . . 9
  - 3.4.4 Calculation and output times . . . 15

4 THAMES Output Files **16**

5 Assumptions **16**

6 Run the Test Case **17**

7 Output File Content **18**
- 7.1 Input data files . . . 18
- 7.2 CSV files . . . 19
- 7.3 Pore size distribution data files . . . 20
- 7.4 Microstructure state files . . . 21
- 7.5 3D rendering file . . . 21

1 Software Requirements

The following software must be installed and running properly on your computer:

- GEMS3K standalone library 3.4.1: http://gems.web.psi.ch/GEMS3K/downloads/index.html
- THAMES v5.1: https://github.tamu.edu/jwbullard/THAMES.git
- (Optional) GEM-Selektor 3.6.0 or 3.7.0: http://gems.web.psi.ch/GEMS3/downloads/index.html
- (Optional) ImageMagick 7 or later (for creating image files in PNG format): https://imagemagick.org/

1.1 Putting THAMES in your path

It will be convenient to add the folder where you install THAMES into your path. Usually the THAMES binary will be located in the `bin` subfolder of your local copy of the git repository that you cloned from GitHub.

Example 1. Suppose the THAMES repository is located at `$HOME/thames`. Open your `$HOME/.bashrc` or your `$HOME/.zshrc` file, depending on whether you use the Bash shell or the Z shell. Somewhere near the bottom of that file, add these two lines:

```
PATH="$PATH:$HOME/thames/bin"
export PATH
```

Save and close this file and then run the command

```
source $HOME/.bashrc
```

or

```
source $HOME/.zshrc
```

You can now test if the system can find the THAMES executable by switching to any other directory and run the command `which thames`. This should output the location of your THAMES file.

Setting up a sandbox to run THAMES simulations

It may be convenient to play around with THAMES by using the test cases provided, but you don't want to pollute your local copy of the repository. Therefore, it is helpful to make copies of all the test cases in a "sandbox". You can run two commands like these to make a sandbox in your home folder:

```
mkdir $HOME/THAMES-Sandbox
cp -R $HOME/thames/tests/* $HOME/THAMES-Sandbox/.
```

Now let's test whether this setup has worked the way we want by running these commands:

```
cd $HOME/THAMES-Sandbox/portcem-298K-sat-wc45
thames
```

This command should produce this:

```
Enter simulation type:
  1) Exit program
  2) Hydration
  3) Leaching
  4) Sulfate attack
```

If you see this output, everything is working correctly and you can `Ctrl-C` to stop.

2 THAMES command line options

Let's stay in that same `portcem-298K-sat-wc45` folder and run this command:

```
thames --help
```

This should show the following output:

```
Usage: "thames [--verbose—v] [--suppress—s] [--xyz—x] [--outfolder—o] folder
[--help—h]"
        --verbose [-v]      Produce verbose output
        --suppress [-s]     Suppress warning messages
        --xyz [-x]          Create 3D visualization movie
        --outfolder [-o]    Name of folder for output data (default is Result)

Note: thames --help [-h] will print this help message
```

This help message is giving a terse description of the different options you can include when running THAMES. The options can be added in any order. Each one is explained a little more fully here.

- `--verbose [-v]`: THAMES already produces significant output to your terminal. This output directly to your screen is called the `stdout` stream, which stands for "standard output". THAMES writes a lot of information to `stdout` while it runs, including a lot of diagnostic messages (in the future much of this will be removed from normal operation but it is helpful while we are still in development mode.) Issuing this option will create even *more* detailed output. You will usually not need or want this output.

- `--suppress [-s]`: This will stop any warning messages from being written to `stdout`. There are not many warning messages, and they can be helpful later to figure out what may have happened if you do not get the results you were expecting. Therefore, it is recommended that you not suppress the warning messages.

- `--xyz [-x]`: This option will cause THAMES to create a file with a `.xyz` extension that can be read directly by a 3D visualization application such as Ovito (available for free download at https://ovito.org). Using this option can cause THAMES to run a little slower because a lot of output is written to this `.xyz` file.

- `--outfolder [-o]`: In addition to the stream of output written to `stdout`, THAMES also creates a large number of new output files, and these are all collected in an

output folder. This option allows you to specify the name and location of this output folder.

- `--outfolder MyFiles` will name the folder `MyFiles` and will put it in the directory where you are running the `thames` command.
- `--outfolder ~/SimFiles` will name the folder `SimFiles` but it will be put in your home folder instead of the current folder
- Omitting this option will name the folder `Results` and it will be put in the directory where you are running `thames`.

2.1 Running THAMES non-interactively

2.1.1 Redirecting input

Running `thames`, with or without any of these options, will enter you into "interactive" mode by default, prompting you with menu options on the command line. When you enter text from your keyboard, you are creating a stream of text called `stdin`, or the "standard input" stream.

In normal operation, THAMES will prompt you for six pieces of information in this order:

1. The type of simulation to run:

   ```
   1) Exit program
   2) Hydration
   3) Leaching
   4) Sulfate attack
   ```

   At the moment options 3 and 4 do not work properly, so you will almost always want to choose "2"

2. The name of the master file with thermodynamic information for GEMS.
3. The name of the file containing the definition of all the microstructure phases that THAMES will recognize for your simulation (JSON format)
4. The name of the file containing the initial 3D microstructure data
5. The name of the file containing the list of times at which to calculate a thermodynamic state or to output microstructure data.
6. The root name that all the output files will have

You can avoid the interactive behavior of THAMES and run a simulation in non-interactive mode by putting all six of your responses in a text file, each on a separate line, and then *redirecting* the input to come from the file instead of fro `stdin`. Each of the test cases in your sandbox have such a text file pre-made for you, called `input.in`.

To run THAMES in non-interactive mode, you just need to issue the `thames` command together with any options, type a space character after the last option, and then type

<input.in &, and then type Return (on Linux or Windows) or Enter on a Mac. So a command that redirects input could be:

```
thames --outfolder MySim < input.in &
```

Incidentally, the < character in a Unix or Linux command always means "redirect input from the file that follows this character". Also that last & character at the end of the command is the Unix way of telling the computer to run the command in the background so you will get your command line prompt back while it is running.

2.1.2 Redirecting output

If you run the previous command, even in background mode, THAMES will still write all of its stdout stream to your screen, effectively making that terminal unusable until it finishes. To avoid this behavior, you should always redirect stdout away from your screen and into a text file to be saved for later. Whereas the symbol for redirecting input is <, the symbol for redirecting all stdout is >. However, if something goes wrong and THAMES issues some kind of error while it is running, that error message will still go to your terminal. You can direct both normal standard output (stdout) and errors (called stderr) to the same text file by using >& (with no space between the two characters) instead of just >.

So finally, a full THAMES command that runs in the background non-interactively would be

```
thames --outfolder MySim < input.in >& output.txt &
```

3 THAMES Input Files

3.1 Categories of THAMES Input

Throughout this document, it will be assumed that your THAMES simulation will happen in a working directory called $WORKDIR.

THAMES requires three categories of input files:

1. The initial 3D microstructure of the material system
2. Thermodynamic data input files produced by GEM-Selektor
3. Microstructure input file and file linking microstructure phases to GEM phases
4. Files to specify calculation times and microstructure output times
5. (Optional) File containing standard input to redirect to the program

Each of these will be detailed in the following sections.

3.2 Microstructure initial 3D arrangement

The microstructure representation is a digitized 3D image with `X_Size` voxels in the x direction, `Y_Size` in the y direction, and `Z_Size` in the z direction. Each voxel is a physical dimension of `Image_Resolution` in micrometer units. This is a normal ASCII text file. The first few lines of an example microstructure are given below.

```
#THAMES:Version: 5.1
#THAMES:X_Size: 100
#THAMES:Y_Size: 100
#THAMES:Z_Size: 100
#THAMES:Image_Resolution: 1.0
1
1
1
2
1
2
1
.
.
.
```

The meaning of the five lines of header information follows:

- `#THAMES:Version` is the THAMES version with which these data should be compatible. It is not currently used in the code but is only for informational purposes;
- `#THAMES:X_Size`, `#THAMES:Y_Size`, and `#THAMES:Z_Size` are the system dimensions, in voxel units, in all three Cartesian directions;
- `#THAMES:Image_Resolution` is the linear dimension of each cubic voxel, in μm units.

After the five-line header to this file, each line contains a single integer corresponding to the ID of one of the microstructure phases (not the GEM phases). The rows have a `z-y-x` nesting convention, by the first voxel is $(0, 0, 0)$, the x coordinates vary most quickly and the z coordinates vary most slowly.

One way to generate such a file is to create a virtual 3D microstructure using VCCTL, which produces a `.img` file in much the same format as just shown. However, the VCCTL phase ID numbers (Table 1) are different than the ones required by THAMES. Therefore, one must replace the VCCTL ID numbers with the corresponding THAMES microstructure id numbers. A C program called `vcctl2thames` has been provided to assist with this, but the program may need to be edited as needed to be consistent with the THAMES microstructure definition file described later in this document.

3.3 Thermodynamic data input

THAMES 5.1 comes with the thermodynamic data files you will need to run most of the simulations you want. This includes systems with temperatures ranging from 283 K to 333 K and including most of the important components of portland cement, portland limestone cement, and oil well cement. It also includes most of the iron sulfides needed for simulations involving pyrrhotite oxidation, as well as some common supplementary cementitious materials (SCMs)

Table 1: VCCTL 9.5 phase identification numbers

Phase	ID	Phase	ID	Phase	ID
Water	0	Gypsum	7	$Ca(OH)_2$	19
Void	55	Bassanite (Hemihydrate)	8	C−S−H	20
Alite	1	Anhydrite ($CaSO_4$)	9	C_3AH_6	21
Belite	2	Silica Fume	10	Ettringite	22
C_3A	3	Inert	11	$Fe(OH)_3$	25
C_4AF	4	Silica glass	16	Pozzolanic C−S−H	26
K_2SO_4	5	Monosulfate	24	Friedel salt	29
Na_2SO_4	6	Monocarboaluminate	34	$CaCl_2$	28
$CaCO_3$	33	Strätlingite	30	Brucite	35

including silica fume and proxies for the main glassy and crystalline phases found in fly ashes. A full list of all the phases included can be found in the appendix.

The thermodynamic data files can be found in any of the various test cases provided with THAMES 5.1, located in subfolders of the `tests` folder:

- `alite-298K-sealed-wc35`
- `highvolume-silicafume`
- `PC-Carbonate-200`
- `PC-FlyAsh-200`
- `portcem-298K-sat-wc45`
- `portcem-298K-sealed-alk-wc45`
- `portcem-298K-sealed-wc45`
- `pyrragg-298K-sat`
- `pyrrcem-296K-sat`

The first seven test cases in this list all have the exact same thermodynamic data files, a collection of five files that all start with `thames-` The last two test cases have data files that contain chlorinated phases such as Friedel's salt and Kuzel's salt, but they do not contain the glassy and crystalline fly ash components.

If you need a system that cannot be modeled with these thermodynamic data files, you will need to use the GEM-Selektor software specified in Section 1 and follow its documentation for creating recipes and exporting data files.

3.3.1 Meaning of GEMS3K the File Contents

For normal usage of THAMES, one does not need to be concerned with most of the contents of the GEMS3K files. However, a few noteworthy items are as follows:

- The `-dat.lst` file contains that string that the GEMS3K library reads when performing a calculation so that it knows which other files contain the thermodynamic data it needs to run.

- The `-dch.dat` file contains the names of the independent components in the ¡ICNL¿ field, the names of the dependent components in the ¡DCNL¿ field, and the names of the phases in the ¡PHNL¿ field. You will need these names when correlating microstructure phases with GEM phases. This file also contains the parameters needed to calculate the molar volumes and densities of the dependent components as a function of temperature and pressure.
- The `-dbr.dat` file contains information about the calculated state of the system calculated using GEM Selektor, such as the activity coefficients, and phase abundances. But these will change with subsequent calculations in THAMES and so they are not of concern in the great majority of cases.
- The `-ipm.dat` file contains information about the numerical parameters used in the calculations, and are not of immediate concern for using THAMES.

3.4 Simulation Parameters

3.4.1 Microstructure phase definition section

THAMES defines a collection of phases that can potentially be present in its 3D microstructure. These are called "(THAMES) microstructure phases," and they can be defined as one or more of the phases recognized in the thermodynamic data files from GEMS. For example, one microstructure phase could be defined for each of the multiple phases in AFm family that GEMS recognizes, or one microstructure phase could be defined as the collection of all of those AFm phases. The user has the liberty to define these microstructure phases, and it is not necessary to include every phase recognized by GEMS within the collection of microstructure phases. If you know that silica fume will never be present within your simulation, it does not need to be defined in any microstructure phase that you create. However, two non-negotiable microstructure phases **must** always be defined: *void space* and *electrolyte* (the aqueous pore solution).

The other information that must be given is the final age of the system to be simulated and the specific times that you want to output microstructure images. These are both given in units of days.

The file for specifying these simulation parameters in a JSON file, which you can name anything you want but which has a default name of `simparams.json`. The documentation for JSON syntax can be found at https://www.json.org. We will examine the typical contents of this file for a portland cement paste with some pyrrhotite included.

3.4.2 Environment section.

The first section is called `environment` and contains information about the curing conditions and any boundary conditions on the electrolyte composition. An example is shown below, with extra annotations that are not included in the actual JSON file

```
{
  "environment": {
    "temperature": 296.15,        # Curing temperature (K), required
    "reftemperature": 298.15,     # Reference temperature (K) for kinetics, required
    "saturated": 1,               # 1 = saturated pores, 0 = sealed pores,
```

```
    "electrolyte_conditions": [   # (optional)
      { "DCname": "Ca(CO3)@", "condition": "initial", "concentration": 1.0e-6 },
      { "DCname": "AlO2H@", "condition": "initial", "concentration": 1.0e-6 },
      { "DCname": "CaSiO3@", "condition": "initial", "concentration": 1.0e-6 },
      { "DCname": "Fe(CO3)@", "condition": "initial", "concentration": 1.0e-6 },
      { "DCname": "MgSiO3@", "condition": "initial", "concentration": 1.0e-6 },
      { "DCname": "KOH@", "condition": "initial", "concentration": 1.0e-6 },
      { "DCname": "Ca(SO4)@", "condition": "initial", "concentration": 1.0e-6 },
      { "DCname": "K+", "condition": "initial", "concentration": 2.0e-6 },
      { "DCname": "SO4-2", "condition": "initial", "concentration": 1.0e-6 },
      { "DCname": "Na+", "condition": "initial", "concentration": 0.672 },
      { "DCname": "ClO-", "condition": "initial", "concentration": 0.672 }
    ],
  },
  .
  .
  .
}
```

The `temperature`, `reftemperature`, and `saturated` lines are required for every simulation. The `electrolyte_conditions` section is techincally optional, but it is a good idea to have it. The GEMS library used by THAMES is more likely to properly converge if there is at least a little bit of each independent component in the electrolyte at the beginning. As shown, this can be done by including small amounts of different dependent components (DCs) that contain the ICs.

The list of all available DCs can be found in the DCNL section of the thermodynamic DCH file (usually called `thames-dch.dat`). The required options for any electrolyte component are:

- `DCname`, must exacdtly match a name in the DCNL section of the `-dch.dat` file.
- `condition` can be "initial" if the component's concentration is to be set initially but can vary as the simulation proceeds, or "fixed" if the component's concentration is to remain at the same value throughout.
- `concentration` is the molal concentration for the component in units of moles per kilogram of water.

Please be aware that the net electrical charge of the electrolyte must be zero, and it is the user's reponsibility to ensure that the concentrations are set in a way to balance the charge. If the charge is **not** balanced, the simulation will throw an exception and exit.

3.4.3 Microstructure phase definitions

The next section is called `microstructure` and defines each of the phases (or collection of phases) that can potentially exist within the 3D microstructure. The characteristics of a microstructure phase definition are

- its name,
- a unique numerical identification number (id) (**Note** that each id appearing in the initial 3D microstructure image file must be equal to the numerical id for one of the microstructure phases defined here),

- whether or not it is considered to be part of the cementitious material or is some other component in the mixture,
- its relation to phases defined in the thermodynamic data,
- its internal, or "subvoxel" porosity (assumed to be zero if not given),
- its internal pore size distribution (optional),
- information about its kinetic behavior (more details to follow below), and
- the color it should have in visualizations produced by THAMES

The section opens with its name (microstructure) and the number of microstructure phases that are defined

```
"microstructure": {
  "numentries": 23,
  .
  .
  .
},
```

In this example there are going to be 23 microstructure phases defined. Remember that every phase in the microstructure must be defined here, but that not every phase defined here must appear in the microstructure. Immediately after the `numentries`, each phase is defined sequentially in a collection named `phases`.

The void phase. The first defined phase *must* be "Void" which is just empty porosity within the microstructure. This phase is unique because it is the only one that does not have any associated thermodynamic phase. The meaning of each entry is

```
  "phases": [
    {
      "thamesname": "Void",
      "id": 0,
      "cement_component": 0,
      "display_data": { "red": 0.0, "green": 0.0, "blue": 0.0, "gray": 0.0 }
     },
     .
     .
     .
  ],
},
```

The meaning of each entry is

- `thamesname` is any string in quotes to designate the phase. The void phase **must** be named `Void` and the pore solution phase **must** be named `Electrolyte`, but you are free to choose the names of all other microstructure phases.
- `id` is the unique numerical identification number, which must be a non-negative integer. The void phase **must** have `id` of 0 and the pore solution **must** have `id` of 1, but you are free to choose the `id` of all other microstructure phases. Just remember that the phase identificatino numbers in the 3D microstructure image file must correspond to the correct microstructure phase `id` here.

- `cement_component` is 1 for any phase that is solid and that belongs to the cementitious material, such as cement clinker phases, gypsum, alkali sulfates, free lime, *etc*, and is 0 for other phases such as void, electrolyte, silica fume, fly ash phases, and any solid hydration products such as C−S−H gel. This distinction is needed for internal calculations of water/cement ratio and water/solids ratio.
- `display_data` is a set of red/green/blue/gray values that specify the color of the phase in THAMES visualizations. Many microstructure phases, though not all, have default colors assigned already according to predefined `thamesname` names, so this entry is only needed if you are defining a new microstructure phase, changing the name of an existing phase, or if you wish to override the default color.

The electrolyte phase. The next phase after the void phase **must** be the pore solution, which has the name `Electrolyte`:

```
{
  "thamesname": "Electrolyte",
  "id": 1,
  "cement_component": 0,
  "gemphase_data": [
    {
      "gemphasename": "aq_gen",
      "gemdc": [
        { "gemdcname": "Al(SO4)+", "gemdcporosity": 1.0 },
        { "gemdcname": "Al(SO4)2-", "gemdcporosity": 1.0 },
        { "gemdcname": "Al+3", "gemdcporosity": 1.0 },
        { "gemdcname": "AlO+", "gemdcporosity": 1.0 },
        { "gemdcname": "AlO2-", "gemdcporosity": 1.0 },
        { "gemdcname": "AlO2H@", "gemdcporosity": 1.0 },
        .
        .
        .
      ],
    }
  ]
}
```

This phase definition starts just like the void phase, with a `thamesname`, a unique `id`, and the designation that it is **not** to be considered a cement component. After these three, there is a section called `gemphase_data` that tells THAMES how to relate this microstructure phase to one or more phases in the thermodynamic system definition (`-dch.dat` file).

- `gemphasename` is the name of the corresponding GEMS system phase found in the PHNL section of the `-dch.dat` file. It must match exactly.
- `gemdc` is a list of all the GEMS dependenct components (DCs) that are assigned to this GEM phase. Each entry has two characteristics:
  - `gemdcname` is the name of the DC, which must actually belong to the GEM phase and which must also exactly match the name in the DCNL section of the `-dch.dat` file. Note that these correspondences of GEM phases and GEM DCs are already set up in the `-dch.dat` file, but they are listed here primarily for solid solutions where the internal pore space of a phase may depend on its composition (see below).

```
## (6) Dependent Components and their codes
# DCNL: Name list of Dependent Components (<=16 characters per name) [nDC]
<DCNL>
'Ca+2' 'CaOH+' 'Ca(HSiO3)+' 'CaSiO3@' 'HSiO3-' 'Si4O10-4' 'SiO2@' 'SiO3-2' 'H2@' 'O2@' 'OH-' 'H+'
'H2O@' 'H2' 'O2' 'H2O' 'CSHQ-JenD' 'CSHQ-JenH' 'CSHQ-TobD' 'CSHQ-TobH' 'C2S' 'C3S' 'Lim'
'Portlandite' 'Qtz' 'Amor-Sl'
# ccDC: Class codes of DCs (Dependent Components) [nDC]
<ccDC>
'S' 'S' 'S' 'S' 'S' 'S' 'S' 'S' 'S' 'S' 'S' 'S' 'T' 'W' 'G' 'G' 'G' 'I' 'I' 'I' 'I' 'I' 'O' 'O' 'O' 'O' 'O'
'O'

# DCmm: Molar masses of DCs, kg/mol [nDC]
<DCmm>
0.0400779991149902 0.0570853492021561 0.117169650197029 0.116161700248718 0.0770916510820389 0.
2723360042572 02 0.060084300994873 0.076083701133728 0.0020158998966217 0.03199880027771 0.
017007350087165 8 0.0010079499483108 5 0.018015300035476 7 0.0020158998966217 0.03199880027771 0.
018015300035476 7 0.169212552442741 0.173886048006892 0.119821407543999 0.124494053130627 0.
172239099502563 0.228316498756409 0.0560773992538452 0.0740926992893219 0.060084300994873 0.
060084300994873

## (7) Phases and their codes
# PHNL: List of Phase names (<=16 characters per name) [nPH]
<PHNL>
'aq_gen' 'gas_gen' 'CSHQ' 'Belite' 'Alite' 'lime' 'Portlandite' 'Quartz' 'Silica-amorph'
# ccPH: Codes of phase aggregate state [nPH]
<ccPH>
'a' 'g' 's' 's' 's' 's' 's' 's' 's'
# nDCinPH: Number of DCs included in each phase [nPH]
<nDCinPH>
13 3 4 1 1 1 1 1 1
```

Figure 1: Portion of a DCH input file showing the definitions of GEM phases and DCs.

- `gemdcporosity` is the volume fraction of non-solid space associated with this DC. It must be a real number on the interval $[0, 1]$.

Note that the electrolyte phase has many DCs, one for each possible dissolved component; only a small fraction of them are shown here for brevity.

The names of all the DC components for a GEM phase must be placed within `gemdc` field, and there must be one of these fields for each GEM phase that you wish to link to the microstructure phase. In addition, the `gemphasename` must exactly match one of the ¡PHNL¿ entries in the `-dch.dat` file, and *all* of the DCs associated with that phase must immediately follow that phase as shown in this example. Determine which GEM DCs are associated with a given phase by consulting the ¡nDCinPH¿ field of the `-dch.dat` file.

To illustrate, Figure 1 shows a portion of a real `-dch.dat` file. In the figure, there are *nine* phases listed in the ¡PHNL¿ field and 26 DCs listed in the ¡DCNL¿ field. The numbers in the ¡nDCinPH¿ field relate the two. Notice that there are *nine* numbers in ¡nDCinPH¿, one for each phase. The ordering of the numbers is the same as the ordering of phase names in the ¡PHNL¿ field. So there are 13 DCs in the first phase, which is `aq_gen`. In addition, those 13 DCs are the *first* 13 DCs in the ¡DCNL¿ list. Moving on, we see that there are three DCs in the next phase, which is `gas_gen`, and those three DCs are the next three after the first 13 in the ¡DCNL¿ list, namely `'H2'`, `'O2'`, and `'H2O'` (the last being the water vapor molecule, not liquid water which is `H2O@`[1]

Solid microstructure phases. Solid microstructure phases may be "simple" (one phase, one DC, and possibly some impurities), "solid solution" (one phase, multiple

[1]In GEMS, the '@' symbol is reserved to designate a zero-charge liquid solvent or a zero-charge dissolved component. Gas molecules do not have this symbol even though they also have zero charge.

DCs that may be present in any mole fraction to give a phase with nonstoichiometric composition, such as C−S−H gel), or "composite" (multiple thermodynamic phases combined for simplicity or computational efficiency).

An example of a "simple" microstructure phase is alite, which is usually a monoclinic form of tricalcium silicate (Ca_3SiO_5) with some impurities to stabilize the monoclinic polymorph.

```
{
  "thamesname": "Alite",
  "id": 2,
  "cement_component": 1,
  "interface_data": { }
  "gemphase_data": [
    {
      "gemphasename": "Alite",
      "gemdc": [{ "gemdcname": "C3S" }]
    }
  ],
  "impurity_data": {
    "k2ocoeff": 0.00087,
    "na2ocoeff": 0.0,
    "mgocoeff": 0.00861,
    "so3coeff": 0.007942
  },
  "kinetic_data": {
    .
    .
    .
  }
},
```

This entry indicates three more features of a solid phase, namely data about its interface with other phases, its impurity content, and its kinetic behavior:

- `interface_data` in the above entry for alite is empty, but can have information about the affinity the phase has for growing on one or more other substrate phases, using the name of the substrate phase and the thermodynamic contact angle between the two:
  - `affinityphase` must be the `thamesname` of another *microstructure* phase defined in this file
  - `contactangle` is the equivalent thermodynamic contact angle for the two phases in units of degrees, and must be a real number on the interval $[0, 180]$, where $0°$ is perfect wetting and $180°$ is complete nonwetting.
- `impurity_data` contains data on the amounts of four different possible impurities, namely potassium, sodium, magnesium, and sulfur, in terms of the *mole fractions* of their equivalent oxides, namely K_2O, Na_2O, MgO, and SO_3. Currently these are the only four impurities that are recognized by THAMES.
- `kinetic_data` is optional and will be described in detail in a later section. For now, the presence of this section indicates that the rates of reaction of this phase will be controlled by a kinetic rate equation. If this section is not provided, the amounts of this phase at any time will be determined solely by thermodynamic equilibrium as calculated by GEMS.

An example of a more complicated composite microstructure phase could be a collection of carbonated AFm phases, which would include all the monocarbonates, hemicarbonates, and tricarboaluminate phases:

```
{
  "thamesname": "AFmc",
  "id": 14,
  "cement_component": 0,
  "interface_data": {
    "affinity": [
      { "affinityphase": "Alite", "contactanglevalue": 180 },
    ]
  },
  "gemphase_data": [
    {
      "gemphasename": "C4Ac0.5H105",
      "gemdc": [{ "gemdcname": "hemicarb10.5" }]
    },
    {
      "gemphasename": "C4Ac0.5H12",
      "gemdc": [{ "gemdcname": "hemicarb" }]
    },
    {
      "gemphasename": "C4Ac0.5H9",
      "gemdc": [{ "gemdcname": "hemicarb9" }]
    },
    {
      "gemphasename": "C4AcH11",
      "gemdc": [{ "gemdcname": "monocarb" }]
    },
    {
      "gemphasename": "C4AcH9",
      "gemdc": [{ "gemdcname": "monocarb9" }]
    }
  ]
},
```

The absence of impurities or kinetic data in this example indicate that the phase is to be considered pure and that its amount in the microstructure at any time will be determined solely by thermodynamic equilibrium with the electrolyte. Also note that each of the DCs associated with this phase default to having zero porosity, so there will be no subvoxel or interhydrate porosity associated with this microstructure phase.

Finally, a solid solution with internal porosity can be defined, such as C−S−H gel:

```
{
  "thamesname": "CSHQ",
  "id": 11,
  "cement_component": 0,
  "interface_data": {
    "affinity": [
      { "affinityphase": "Alite", "contactanglevalue": 30 },
      { "affinityphase": "AFmc", "contactanglevalue": 90 },
    ]
  },
  "gemphase_data": [
    {
      "gemphasename": "CSHQ",
      "gemdc": [
        { "gemdcname": "CSHQ-JenD", "gemdcporosity": 0.4935 },
        { "gemdcname": "CSHQ-JenH", "gemdcporosity": 0.4935 },
        { "gemdcname": "CSHQ-TobD", "gemdcporosity": 0.2004 },
        { "gemdcname": "CSHQ-TobH", "gemdcporosity": 0.2004 },
        { "gemdcname": "KSiOH", "gemdcporosity": 0.1825 },
        { "gemdcname": "NaSiOH", "gemdcporosity": 0.1825 }
```

```
        ]
      }
    ],
    "poresize_distribution": [
      { "diameter": 0.23228, "volumefraction": 0.01684781722375019 },
      { "diameter": 0.49089, "volumefraction": 0.02276126023357459 },
      .
      .
      .
    ],
    "Rd": [
      { "Rdelement": "K", "Rdvalue": 0.42 },
      { "Rdelement": "Na", "Rdvalue": 0.42 }
    ]
  },
```

Two more sections can be found in this example. First, the pore size distribution can be defined as a normalized probability density function, with the differential volume fraction (`volumefraction`, dimensionless) given for each effective pore size dimension (`diameter`) given in nanometer units. The second new section is the impurity partition section, with the name `Rd`, that describes the proportion of impurities that will be drawn from the surrounding electrolyte if available:

- `Rdelement` is the name of an independent component (IC) found in the `ICNL` section of the `-dch.dat` file.
- `Rdvalue` is the partition coefficient defined for alkalis by Hong and Glasser (*Cem Concr Res*, 29 (1999), 1893–1903; *Cem Concr Res*, 32 (2002), 1101-1111.) which is defined

$$R_\mathrm{d} = \frac{c_s w}{c_d s} \qquad (\text{units of } \mathrm{L\,kg^{-1}}) \tag{3.4.1}$$

  where c_s is the concentration of the impurity in the solid phase (units $\mathrm{mol\,L^{-1}}$), c_d is the concentration of the impurity in the electrolyte (units $\mathrm{mol\,L^{-1}}$), and w/s is the water-solids ratio in units of $\mathrm{L\,kg^{-1}}$

3.4.4 Calculation and output times

The final section of this file is called `time_parameters` and defines both the final age of the material to simulate and the simulation times at which THAMES should write data for visualization and for assessing the pore size distribution and saturation state.

```
"time_parameters": {
  "finaltime": 56.0,
  "outtimes": [0.01, 0.1,0.5,1.0,3.0,7.0,14.0,28.0]
}
```

Or, alternatively,

```
"time_parameters": {
  "finaltime": 56.0,
  "outfreq": 4.0,
}
```

The meaning of the parameters are as follows:

- `finaltime` is the final time, in days, to attempt simulating the microstructure evolution. For various reasons the simulation may not complete the full requested age—for example, the microstructure could be completely depleted of electrolyte and so not be able to perform any other reactions—but the simulation will definitely stop after this time is reached.
- `outtimes` is a list of times, in days, at which THAMES should output the microstructure state, which includes
  - the full 3D microstructure image (an `.img` file),
  - a 2D image of a slice of the microstructure (both `.ppm` and `.png` files), and
  - An entry in the `.xyz` file used for 3D visualizations.[2]
  - the current pore size distribution and saturation state.
- `outfreq` is an alternative to `outtimes` offered for convenience. It allows one to specify a time interval, in days, at the end of which the output images and pore size distributions will be output. This enables one to avoid entering many equally spaced times for output. Note: If both `outfreq` and `outtimes` are present, the `outtimes` entry will be ignored.

4 THAMES Output Files

This section provides guidance on the information contained in the various output files produced by THAMES v5.1. For simplicity, the guide will be structured entirely in the form of an example simulation based on the test case named `portcem-296K-sealed-alk-wc45` that is provided.

5 Assumptions

This guide assumes

1. that you have been through the previous sections of this document,
2. that your THAMES 5.1 distribution located in a directory with the absolute path specified in this guide by `$THAMES`. For example if your distribution is located at the absolute path `/home/odehwah/models/THAMES`, then `$THAMES` is equivalent to that full path in this guide; and
3. that you have already built `thames` and that it is in your `$PATH` environment variable so that the operating system can discover it when you run it from the command line. If you are unsure whether this has been done, you can run the command `which thames` from the command line. If that command returns something like `thames not found`, then you must first augment your `$PATH` environment variable. If you are unsure how to change the environment variable a tuto-

[2] The 3D visualization option is discussed more in Section 7.5.

rial for Linux can be found here: https://www.digitalocean.com/community/tutorials/how-to-view-and-update-the-linux-path-environment-variable

6 Run the Test Case

A good idea is to keep the working directory as clean as possible so that it doesn't get accidentally corrupted or more confusing. So it is a good practice to copy the test case to another location before using it there. For this example, create a temporary directory and then copy the test case there:

```
mkdir $HOME/scratch
cp -R $THAMES/test/portcem-296K-sealed-alk-wc45 $HOME/scratch
```

Next, go to that new directory:

```
cd $HOME/scratch/portcem-296K-sealed-alk-wc45
```

Before running the simulation, open the `input.in` file. The file contents are:

```
2
thames-dat.lst
simparams.json
ccr140-w45-thames.img
cem140-sealed-alk-01
```

Each line of this file is now explained:

- **Line 1**: Determines the type of simulation to be run:
  1. A value of 1 exits the simulation with no further action
  2. A value of 2 will run a normal hydration simulation
- **Line 2**: The root name of the thermodynamic data. This file is read and automatically prompts the input of the other thermodynamic data files.
- **Line 3**: The JSON formatted input file specifying the simulation environment, the microstructure phase definitions, and the simulation time and output frequency.
- **Line 4**: The name of the file containing the initial 3D microstructure
- **Line 5**: The root name for all output files produced by THAMES. This can be anything you want, although it is recommended not to use periods within it. Feel free to shorten it to `cem140` or some other string if you wish.

Close the file, making sure to save it first if you made changes to the fifth line, and then run the THAMES simulation with these options:

```
thames --xyz --outfolder MyResult < input.in >& output.out &
```

The simulation should create an `output.txt` file and a folder named `MyResults`, as shown in Figure 2, while it continues to run for about five minutes to ten minutes.

A message similar to the following appear in the terminal when the simulation finishes:

Figure 2: Contents of the test directory running a simulation.

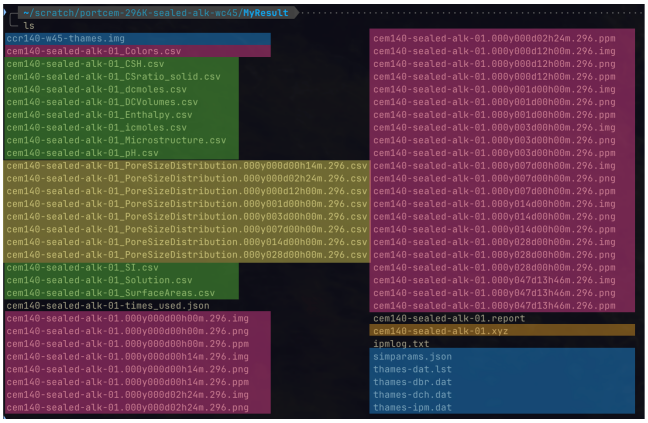

Figure 3: Contents of the `MyResult` folder at the conclusion of THAMES simulation. Color blocks superimposed on the files indicate spreadsheet-like CSV data (green), pore size distribution data (yellow), microstructure data and images for viewing (pink), a 3D `xyz` format file for viewing in Ovito or similar application (orange, optional), and original input data files (blue) copied from the parent directory for record-keeping.

```
[1]  + 56549 exit 1     ~/Software/THAMES/bin/thames --xyz --outfolder MyResult < input.in >&
```

The `MyResults` folder holds all of the output files created by the simulation, while the `output.txt` file holds all the standard output that would otherwise be written to the screen. Under ordinary circumstances there is no need to examine the `output.txt` file, but it can be useful to have it if the program crashes for some reason or otherwise behaves oddly.

7 Output File Content

When the simulation exits, navigate to the `MyResults` folder and run the `ls` command to see the directory contents, which should appear like that shown in Fig. 3.

7.1 Input data files

The files highlighted in blue in Fig. 3 are input files copied directly from the parent directory and are present only as metadata to recall the conditions under which the simulation was performed. The contents of most of these files are described in detail in the companion document on preparing input files and will therefore not be revisited

here.

7.2 CSV files

Eleven comma separated value CSV files are written by THAMES to recored time-dependent simulation data. These are highlighted in green in Fig. 3 and are described in the following paragraphs:

_CSH.csv Records the composition of the calcium silicate hydrate (C−S−H) phase as a function of time. The first column is the time (in hours). The rest of the columns up until the final column are the concentrations of each element in the C−S−H phase. The final column displays the molar Ca/Si ratio of the C−S−H.

_CSratio_solid.csv Provides the overall molar Ca/Si ratio of all solid components in the microstructure. Time (in hours) is in the first column and the only other column is the averaged Ca/Si ratio. This file may be removed in future versions of THAMES.

_dcmoles.csv Catalogs the number of moles (per 100 g of initial solid) of each dependent component (DC)[3] in the system, again with simulation time (in hours) occupying the first column.

_DCVolumes.csv Provides the volume (in m^3 per 100 g of initial solid) of each dependent component (DC) in the system, again with simulation time (in hours) occupying the first column.

_Enthalpy.csv Displays in the second column the predicted cumulative heat release by the system (in J per 100 g of initial solid) at each corresponding time (in hours) in the first column.

_icmoles.csv Gives the calculated moles, per 100 g of initial solid, of each independent component (IC, essentially chemical elements and electric charge) in the system for each corresponding time (in hours) in the first column. This file may be removed in future versions of THAMES because it duplicates information that can be easily obtained from the `_dcmoles.csv` file.

_Microstructure.csv Records the *volume fraction* (dimensionless) of each defined microstructure phase in the system at the corresponding time (in hours) displayed in the first column. The very last column gives the predicted chemical shrinkage value at the corresponding time, in units of m^3 per 100 g of initial solid material.

[3]Note that DCs are not identical with phases. Some phases, such as gypsum, have only one DC, which for gypsum is named Gp, while phases such as C−S−H are solid solutions of multiple DCs.

_SI.csv Contains the saturation index of the pore solution (dimensionless) with respect to each defined microstructure phase at each corresponding time (in hours) in the first column. Here the saturation index, Ω, is defined as the ratio of the activity product to the equilibrium constant, so for any given solid phase p, $\Omega_p = 1$ if the solution is at equilibrium with that solid phase, $\Omega_p < 1$ if the solution is undersaturated so that the phase p is thermodynamically driven to dissolve, and $\Omega_p >$ if the solution is supersaturated so that the phase p is thermodynamically driven to precipitate.

_Solution.csv Displays the composition of the electyrolyte (*i.e.*, pore solution) at each calculated time (in hours) given in the first column. The composition consists of the concentration of each dependent component dissolved in the water, given in units of $\mathrm{mol\,kg^{-1}}$ water.

_SurfaceAreas.csv Displays the solid-water surface area of each defined microstructure phase at each calculation time (in hours) displayed in the first column. The surface areas in this file are given in units of $\mathrm{m^2}$ per 100 g of initial solid material. In particular, they are not equivalent to the specific surface area of each phase individually, but come closer to reflecting each phase's contribution to the specific surface area.

7.3 Pore size distribution data files

One of these files is created for each output time the user specifies in the input JSON file, which is described in more detail in the companion guide for preparing input files. Each file has a name that contains the simulation temperature in kelving units and the time in year-day-hour-minute format. For example, a file with the name

```
myfile_PoreSizeDistribution.001y.056d.14h.30m.296K.csv
```

corresponds to a temperature of 296 K and a simulation time of 1 year, 56 days, 14 hours, and 30 minutes. Times are always rounded to the nearest minute.

The contents of a pore size distribution file are structured as shown below:

```
Time = 672 h,,
Voxel-scale pore volume fraction (>= 1000 nm) = 0.049738,,
Saturated voxel-scale pore volume fraction = 0,,
Empty voxel-scale void volume fraction = 0.049738,,
Subvoxel volume fraction (< 1000 nm) = 0.173083,,
Saturated subvoxel volume fraction (< 1000 nm) = 0.13513,,
Empty subvoxel-scale void volume fraction = 0.0379536,,
Total pore volume fraction = 0.222821,,
Total void volume fraction = 0.049738,,
Pore size saturation data:,,
Masterporevolume size = 56,,
Diameter (nm),Volume Fraction,Fraction Saturated
1.99526,0.0426508,1
2.23872,0.00530618,1
2.51189,0.00477843,1
2.81838,0.00436585,1
    .
    .
    .
63.0957,0.00507889,1
70.7946,0.00853574,0.345385
79.4328,0.00906405,0
89.1251,0.00953302,0
```

```
100,0.0115934,0
223.872,0.0021721,0
501.187,3.43993e-06,0
>=1000,0.049738,0
```

The first eleven lines provide overall data on the volume fraction of different ranges of saturated pores and empty pores. Following this is the more detailed analysis of pore volume fractions. The first column is the effective pore diameter in nm units, the second column is the volume fraction (dimensionless) of the microstructure occupied by pores of that size, and the third column is the fraction of that pore volume occupied by the pore solution. The data are provided up to a maximum pore diameter of 99 nm, and all larger pores are lumped into a final category on the final row.

7.4 Microstructure state files

For each specified output time in the JSON input file, two or three files will be created:

- `.img` files contain the full 3D microstructure image data in exactly the same format as the input microstructure image, which is discussed in detail in the companion guide for preparing input files.
- `.ppm` is a 2D xy-slice through the microstructure at the midpoint of the z dimension. The format is a color portable pixel map (PPM) that can be displayed by a range of image rendering applications such as Gimp (any platform, most flexible), `eog` or `feh` (linux), Preview (Mac OS), Honeyview or i_view (Windows).
- `.png` is a PNG version of the corresponding `.ppm` file that THAMES will generate automatically only if ImageMagick 7 or later https://imagemagick.org/ is installed on the computer and if the `magick` command is in the path.

The `.ppm` and `.png` files are created with some sense of depth by making the pore space slightly transparent. These two files also render each microstructure phase with a unique color. The `rgb` triplet of values for the color of each phase is given in the `_Colors.csv` file for easier reference.

7.5 3D rendering file

This file tends to be quite large and so it is only created if the user gives the `--xyz` option on the command line when starting a THAMES simulation. It contains a full 3D data structure for each output time specified in the JSON input file, and so can be rendered into individual states or produced into an animation of the microstructure's time evolution. The file is written in the industry-standard `.xyz` format that can be read by multiple 3D rendering software applications. Among these, Ovito (https://ovito.org)[4] is a freely available application that runs on any platform and is relatively easy to use.

[4]Mention of commercial products is for information only, and does not imply recommendation or endorsement by Texas A&M University.